📷 cordova-plugin-mlkit-barcode-scanner

Purpose of this Project

The purpose of this project is to provide a barcode scanner utilizing the Google ML Kit Vision library for the Cordova framework on iOS and Android. The MLKit library is incredibly performant and fast in comparison to any other barcode reader that I have used that are free.

Plugin Dependencies

Dependency	Version	Info
`cordova`	`>=11.0.0`	
`cordova-android`	`>=14.0.0`	compileSdk 35+, CameraX 1.4 (16 KB page size)
`cordova-ios`	`>=7.0.0`	GoogleMLKit pod 9.0, iOS 15.5+

`cordova-plugin-androidx` and `cordova-plugin-androidx-adapter` are not needed anymore.

On iOS the ML Kit pods require iOS 15.5 or higher. If needed, set it in your `config.xml`: `<preference name="deployment-target" value="15.5" />`

Some of the pods ML Kit depends on declare a deployment target below iOS 15, which current Xcode versions reject. The plugin includes a hook (`hooks/ios-pods-deployment-target.js`) that raises them to 15.0 automatically on `cordova prepare` and `cordova build`. If you build straight from Xcode after a fresh `pod install`, run `cordova prepare ios` once first.

On iOS the camera usage text can be set at install time: `cordova plugin add https://github.com/remoorejr/cordova-plugin-mlkit-barcode-scanner.git#dev --variable CAMERA_USAGE_DESCRIPTION="Scan barcodes"`

Installation

Run this command in your project root:

```
npx cordova plugin add https://github.com/remoorejr/cordova-plugin-mlkit-barcode-scanner.git#dev
```

See CHANGES.md for everything that changed compared to version 3.x.

Supported Platforms

- Android
- iOS/iPadOS

Barcode Support

1d formats	Android	iOS
Codabar	✓	✓
Code 39	✓	✓
Code 93	✓	✓
Code 128	✓	✓
EAN-8.	✓	✓
EAN-13	✓	✓
ITF	✓	✓
MSI	✗	✗
RSS Expanded	✗	✗
RSS-14	✗	✗
UPC-A	✓	✓
UPC-E	✓	✓

2d formats	Android	iOS
Aztec	✓	✓
Codablock	✗	✗
Data Matrix	✓	✓
MaxiCode	✗	✗
PDF417	✓	✓
QR Code	✓	✓

ℹ️ Note that this API does not recognize barcodes in these forms:

- 1D Barcodes with only one character
- Barcodes in ITF format with fewer than six characters
- Barcodes encoded with FNC2, FNC3 or FNC4
- QR codes generated in the ECI mode

Usage

To use the plugin simply call `cordova.plugins.mlkit.barcodeScanner.scan(options, sucessCallback, failureCallback)`. See the sample below.

```
cordova.plugins.mlkit.barcodeScanner.scan(
  options,
  (result) => {
    // Do something with the data
    alert(result);
  },
  (error) => {
    // Error handling
  },
);
```

Plugin Options

The default options are shown below. All values are optional.

Note that the `detectorSize` value must be between `0` and `1`, because it determines how many percent of the screen width should be covered by the scan area. If the value is greater than 1 the scan area will not be visible on the screen.

The scan area is drawn as a white rectangle with a red horizontal line through its vertical center and a small red tick at the center of that line, as a visual targeting aid. Only barcodes inside the scan area are read.

```
const defaultOptions = {
  barcodeFormats: {
    Code128: true,
    Code39: true,
    Code93: true,
    CodaBar: true,
    DataMatrix: true,
    EAN13: true,
    EAN8: true,
    ITF: true,
    QRCode: true,
    UPCA: true,
    UPCE: true,
    PDF417: true,
    Aztec: true,
  },
  beepOnSuccess: false,
  vibrateOnSuccess: false,
  detectorSize: 0.6,
  rotateCamera: false,
  captureButton: false,
  detectorHeight: 0,
  highlightBarcode: false,
  showCaptureButton: false,
};
```

By default a barcode is returned as soon as one is detected in the scan area. The options `captureButton` and `highlightBarcode` change this so that the user decides which barcode is returned.

`captureButton`: when `true`, barcodes are no longer scanned automatically. The user lines up the barcode in the viewfinder and taps the scan area (or the capture button, see `showCaptureButton`) to scan it. If no barcode is found, a short message is shown and the user can try again.

`detectorHeight`: height of the scan area. `detectorSize` is the width of the scan area as a fraction of the screen width, `detectorHeight` its height as a fraction of the smaller screen dimension (e.g. `detectorSize: 0.9` and `detectorHeight: 0.3` gives a wide, flat rectangle). In landscape (Android and iOS) the scan area becomes wider with the screen but keeps its portrait height. The default `0` means "same as `detectorSize`" (a square in portrait). Only barcodes inside the scan area are read.

`highlightBarcode`: when `true`, all barcodes inside the scan area are outlined and the one that will be returned is highlighted in green and its value is shown above it. By default it is the barcode closest to the center of the scan area. If it is not the right one, tap another outlined barcode to select it instead. Tap the selected barcode, or anywhere else in the scan area, (or the capture button, if `showCaptureButton` is `true`) to return the selected barcode. Example for a page full of barcodes:

```
cordova.plugins.mlkit.barcodeScanner.scan(
  { detectorSize: 0.9, detectorHeight: 0.35, highlightBarcode: true },
  success,
  failure,
);
```

`showCaptureButton` (default `false`): whether a capture button is shown at the bottom of the scanner when `captureButton` or `highlightBarcode` is enabled. Without it, capture by tapping the scan area.

Output/Return value

```
result: {
  text: string;
  format: string;
  type: string;
}
```

Known Issues

On some devices the camera may be upside down.

Here is a list of devices with this problem:

- Zebra MC330K (Manufacturer: Zebra Technologies, Model: MC33)

Current Solution: if your device has this problem, you can call the plugin with the option `rotateCamera` set to `true`. This will rotate the camera stream by 180 degrees.

Development

Build Process

This project uses npm scripts for building:

```
# removes the generated www folder
npm run clean

# build the project (TypeScript to www/)
npm run build
```

The built `www/` folder is committed to the repository so the plugin can be installed straight from git. Run `npm run build` and commit `www/` after changing anything in `src-typescript/`.

Run the test app

Install cordova:

```
npm i -g cordova
```

Go to test app:

```
cd test/scan-test-app
```

Install node modules:

```
npm i
```

Prepare Cordova:

```
cordova prepare && cordova plugin add ../../ --link --force
```

Build and run the project Android:

```
cordova build android && cordova run android
```

and iOS:

```
cordova build ios && cordova run ios
```

Versioning

⚠️ Before incrementing the version in `package.json`, remember to increment the version in `plugin.xml` by hand.

VS Code Extensions

This project is intended to be used with Visual Studio Code and the recommended extensions can be found in `.vscode/extensions.json`. When you open this repository for the first time in Visual Studio Code you should get a prompt asking you to install the recommended extensions.